Name: NII OBODAI SQUIRE

Student ID: 22425147

GitHub Repository: https://github.com/FlaviusOBO/Frozen-Lake-from-First-Principles-Using-Q-Learning-

UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

DSCD 614 — REINFORCEMENT LEARNING (SEMESTER II 2025/2026)

SOLVING 8×8 FROZEN LAKE FROM FIRST PRINCIPLES USING TABULAR Q-LEARNING

1. INTRODUCTION

Reinforcement Learning (RL) defines an algorithmic paradigm where an autonomous agent learns optimal decision-making frameworks through trial-and-error interactions with a dynamic environment. Operating under the mathematical structure of a Markov Decision Process (MDP), the agent continuously observes environmental states, selects actions based on an underlying behavioral policy, and receives localized scalar rewards alongside state transitions. The objective is to discover a policy that maximizes the expected cumulative discounted return over an infinite or finite timeline.

The Frozen Lake environment serves as a fundamental grid-world benchmark designed to test an agent's ability to navigate under sparse reward regimes while bypassing high-risk operational traps. In this assignment, an 8×8 layout containing a starting position (S), safe frozen tiles (F), hazardous terminal holes (H), and a target goal tile (G) is implemented entirely from first principles. To strictly adhere to academic directives, no external frameworks such as OpenAI Gym, Gymnasium, or Stable Baselines were employed. This technical report details the clean, decoupled architectural separation between the customized environment engine (***environment.py***) and the core tabular updates engine (***agent.py***), accompanied by comprehensive training convergence validations.

2. ENVIRONMENT DESIGN

The custom grid-world environment is encapsulated inside the class ***FrozenLakeEnv*** within ***environment.py***. The physical map configuration precisely mimics the rectangular 8×8 layout specified in the assignment documentation.

2.1 State and Action Representations

To preserve memory localization and facilitate direct table lookup sequences, spatial positions are flattened into a single integer state index map bounded in the interval ***[0, 63]***. The conversion between continuous grid dimensions and row/column coordinates is handled through the following mapping relationship:

$$state\_index = row \times n\_cols + col$$

The action space is discrete, offering four cardinal directional choices represented by specific numerical codes: Left (0), Down (1), Right (2), and Up (3). Boundary condition clipping is explicitly enforced within the state transitions layout; any movement vector that would push the agent outside the 8×8 boundary parameters forces the agent to remain in its pre-step cell index.

2.2 Reward Structure and Terminal States

The environmental feedback structure is highly sparse, presenting clear optimization hurdles. An agent traversing safe frozen paths (***F***) or beginning at the start state (***S***) receives a reward scalar of ***0.0***. Reaching a terminal hole state (***H***) instantly triggers an episode termination flag with a reward of ***0.0***. Conversely, successfully arriving at the goal coordinate (***G***) triggers termination accompanied by a positive scalar reward of ***+1.0***.

3. Q-LEARNING ALGORITHM

Tabular Q-Learning operates as a model-free, off-policy, value-based Temporal Difference (TD) control routine designed to iteratively converge upon the optimal action-value function $Q^*(s, a)$.

3.1 Core Update Mechanism

The underlying Q-table initializes as a zero-filled array of shape $(64, 4)$, providing a neutral baseline across all state-action coordinates. Upon experiencing an interactive state transition tuple (s, a, r, s'), the corresponding cell is updated using the temporal-difference error formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where α represents the learning rate regulating step sizes, and γ signifies the temporal discount factor. For terminal transitions—such as stepping into a hole or arriving at the final target—the bootstrapping prediction factor $\max_{a'} Q(s', a')$ is omitted to avoid calculating post-terminal rewards.

3.2 Exploration vs. Exploitation Management

To balance the exploration-exploitation dilemma, an exponential ε-greedy behavioral policy is introduced. Actions are chosen via:

With probability ε: Choose a random action $a \in A$
With probability $1 - \varepsilon$: Choose $\operatorname{argmax}_a Q(s, a)$

To prevent structural directional biases when multiple fields evaluate to identical values (common during early episodes), uniform random tie-breaking is handled using ***np.flatnonzero***.

4. TRAINING METHODOLOGY

The agent undergoes training over a duration of 5,000 episodes, capped at a maximum threshold of 200 steps per episode to prevent endless loops. Hyperparameter selections are summarized in Table 1 below.

Table 1: Training Hyperparameter Configuration

Hyperparameter Metric	Selected Value	Functional Description
Total Episodes	5000	Total training episodes completed
Max Steps / Episode	200	Truncation ceiling preventing infinite wandering loops
Learning Rate (α)	0.1	Temporal difference update step size scaling Factor
Discount Factor (γ)	0.99	Weighting modifier prioritizing long-term scalar goals
Initial Epsilon (ε_{start})	1.0	Initial random exploration probability coefficient
Minimum Epsilon (ε_{min})	0.01	The mandatory exploitation exploratory baseline floor
Epsilon Decay Rate	0.9995	Multiplicative attenuation coefficient applied per episode
Randomization Seed	42	Base integer anchor guaranteeing deterministic execution

5. EXPERIMENTAL RESULTS AND DISCUSSION

The runtime telemetry gathered across the training duration reveals a smooth learning progression. Telemetry snapshots logged every 500 episodes highlight an accelerating learning trajectory:

- **Episodes 1 – 1,000:** The agent experiences high failure rates, with moving success tracking near 0.0% to 20.0%, driven by heavy exploration (ε decays from 1.0 down to ~0.60).
- **Episodes 1,500 – 3,000:** Success rates rise from 50.0% to over 80.0% as the agent discovers stable high-reward paths around dangerous holes.
- **Episodes 4,000 – 5,000:** Learning stabilizes as the exploration probability nears its minimum floor ($\varepsilon \rightarrow 0.01$). The rolling training success rate plateaus between 92.0% and 96.0%.

Following training, independent validation runs spanning 100 continuous greedy exploitation episodes ($\varepsilon = 0$) achieved a **100.00% Success Rate** with an Average Reward of **1.0000** and zero operational failures.

6. LEARNED POLICY VISUALIZATION

Extracting the argmax values from the trained Q-table provides a complete directional map across the non-terminal grid positions. The generated policy matrix is displayed in the grid below:

```
↓  ↓  ↓  ↓  ↓  ↓  ↓  ↓
→  →  →  →  →  ↓  ↓  ↓
↑  ↑  ↑  H  →  →  ↓  ↓
↑  ↑  ↑  H  →  →  →  ↓
↑  ↑  ↑  H  →  →  →  ↓
↑  H  H  →  →  ↑  H  ↓
↑  H  ←  ←  H  ↓  H  ↓
←  ←  ←  H  ←  →  →  G
```

The learned arrows highlight a safe corridor routing around the central row holes and navigating down the right-hand column toward the goal.

7. CHALLENGES ENCOUNTERED

Two primary technical hurdles were resolved during implementation:

1. **Standard Argmax Bias:** Default implementations of ***np.argmax*** always return the first index during ties. This creates directional biases early in training when Q-values are mostly zero. Implementing an explicit random tie-breaker using boolean masking resolved this issue.
2. **Sparse Reward Signal:** Since the agent only receives feedback upon hitting the goal, early exploration steps yield no gradients. Fine-tuning the decay modifier to ***0.9995*** maintained exploration long enough to discover the goal.

8. CONCLUSION

This assignment successfully demonstrates tabular Q-learning on an 8×8 grid-world from scratch. By decoupling the environment logic from the policy mechanics, the model achieved a 100.0% evaluation success rate without external RL frameworks. Future iterations will explore stochastic settings to evaluate policy robustess under uncertainty.